

OS - ASSIGNMENT - CS & IT DEPARTMENT

SUBMISSION DATE: THURSDAY, 30 APRIL, 2026
DATE: THURSDAY, 28th MAY 2026
MODULE CODE: 351 CS 2104
MODULE NAME: OPERATING SYSTEMS
SEMESTER: III

1. Assignment Overview & Objectives

You will design, build, and document **EduOS** — a multi-component simulator that demonstrates real operating-system concepts through working code. Unlike a standard programming exercise, EduOS requires you to *integrate two languages, track your progress publicly on GitHub, defend design decisions in writing, and produce measurable, reproducible outputs.*

Objective:

The main objective of the assignment is to introduce students to version control and allow students to deep dive into operating system implementation and to be exposed to concept that may not be explicitly found within the Operating systems syllabus. The assignment has it's prerequisites as **Python programming as well as Problem Solving Using C** which will be used in the development of this assignment.

NB: **All concepts that students are unfamiliar will require students to study and implement to the best of their abilities. As such this assignment also present a great learning opportunity that students must fully utilize.**

Part 1: GitHub Repository Setup

10 Marks | Mandatory — late or missing GitHub = automatic 0 on all parts

1.1 Repository Requirements

- **Name:** [OperatingSystem Assignment](#)
- **Visibility:** Public OR private with instructor added as collaborator before the deadline.
- **Collaborator email:** crvsadechiwalo004@gmail.com
- **README.md** (must include all of the following):
 - Project title, Module code, student name, and Registration Number (≤ 120 words)
 - Prerequisites and step-by-step build instructions for both C and Python parts
 - Annotated directory tree (explain every folder/file)
 - At least three labelled screenshots showing the simulator running
 - **output** — paste the final summary line (e.g., "0 errors from 0 contexts") verbatim
 - Challenges encountered and how you resolved each one

OS - ASSIGNMENT - CS & IT DEPARTMENT

- References / attributions (course notes, man pages, books, online resources)
- **.gitignore** covering at minimum: *.o, __pycache__, *.pyc, venv/, *.out, .DS_Store
- **Commit history:** minimum 10 meaningful commits with descriptive messages.

1.2 Recommended Directory Structure

```
EduOS-[StudentID]/ |—— README.md |—— .gitignore |—— docs/ | |—— report.pdf | |——  
screenshots/ |—— c_core/ | |—— Makefile | |—— include/ | |—— eduos.h ← shared  
struct definitions | |—— process_manager.c | |—— thread_manager.c | |—— ipc_module.c | |——  
— main_sim.c |—— python_scheduler/ | |—— scheduler_sim.py | |—— gantt.py | |——  
sample_processes.csv | |—— requirements.txt |—— controller/ |—— main_controller.py ←  
integration bridge
```

Part 2: Core Simulator in C (Low-Level)

35 Marks | **Must compile with:** `gcc -Wall -Wextra -pthread -o eduos *.c` (zero warnings required)

2.1 Process Control Block & process_manager.c

Define the following struct exactly as shown; you may add fields but may NOT remove any:

```
typedef struct { pid_t pid; /* unique process ID */ char name[64]; /* process name  
/ program */ int state; /* NEW | READY | RUNNING | WAITING | TERMINATED */ int  
priority; /* 0 = highest */ int burst_time; /* total CPU time needed */ int  
arrival_time; /* clock tick of arrival */ int remaining_time; /* used by preemptive algorithms */  
int memory_req_kb; /* memory footprint in KB */ int thread_count; /* threads spawned by  
process */ time_t creation_time; /* wall-clock timestamp */ } PCB;
```

Implement the following system-call-like functions (each must print a timestamped log line to stdout):

- `pid_t edu_fork(PCB *parent)` — creates a child PCB by copying the parent; assigns a new PID from a monotonic counter; sets state to NEW then READY.
- `void edu_exec(pid_t pid, char *prog_name)` — replaces a process's program image (name field) and resets burst/remaining times; simulates the UNIX `exec()` contract.
- `int edu_wait(pid_t parent_pid)` — blocks the calling process until all child PCBs in the table reach TERMINATED state; return value is the child's exit code.
- `void edu_exit(pid_t pid, int exit_code)` — marks the PCB as TERMINATED, records exit code, frees dynamically allocated resources.
- `void edu_ps()` — prints a formatted table of all active PCBs (similar to the Linux `ps aux` command).

OS - ASSIGNMENT - CS & IT DEPARTMENT

PCB Serialisation

Your `process_manager.c` must write the full PCB table to a JSON file (`pcb_snapshot.json`) every time a state change occurs. The Python controller reads this file to feed the scheduler. You must implement the JSON serialiser yourself in C — do NOT use an external JSON library.

2.2 Thread Pool & `thread_manager.c`

Implement a configurable thread pool using POSIX pthreads. The pool must:

- Maintain a fixed-size array of worker threads (size set via `#define THREAD_POOL_SIZE 4`)
- Use a **task queue** protected by a mutex + condition variable (`pthread_mutex_t` / `pthread_cond_t`).
- Support graceful shutdown: all threads must finish current tasks before exiting.

Threading Models Demonstration

Simulate and document (with comments and README explanation) how each model works in EduOS:

- **Many-to-One:** All user-level EduOS threads map to a single kernel thread. Implement using `setjmp/longjmp` or a simple cooperative scheduler that you write. Explain the blocking limitation.
- **One-to-One:** Each EduOS thread maps 1-to-1 to a POSIX pthread. Demonstrate the concurrency advantage using a parallel summation task.

Mandatory Race Condition Demonstration

You MUST demonstrate this in two clearly labelled compilation targets:

- **make race** — compiles `thread_manager.c` WITHOUT mutex protection. Multiple threads increment a shared counter. Run the program 5 times and record that the results are non-deterministic (show output in README).
- **make fixed** — compiles WITH `pthread_mutex_t` protection. Show that the counter is now always correct. Additionally add a semaphore-based producer-consumer variant using `sem_init`.

Also identify and fix at least one **deadlock scenario** (create two mutexes acquired in opposite order, detect the deadlock, then resolve it with a consistent lock ordering policy).

2.3 Interprocess Communication

Implement BOTH of the following IPC mechanisms:

- **POSIX Shared Memory** (`shm_open`, `mmap`) — two processes write and read a shared struct containing process metrics. Use a mutex to coordinate access.

OS - ASSIGNMENT - CS & IT DEPARTMENT

- **Anonymous Pipe** (`pipe()`) — a parent process sends serialised PCB data to a child process via a pipe. The child parses and prints the data.

Provide a short driver (`main_sim.c`) that exercises both IPC mechanisms and prints results to stdout for the Python controller to consume.

2.4 C Code Quality Requirements

- All files must pass: `gcc -Wall -Wextra -pthread -std=c11` with zero warnings.
- **Memory management:** Run `valgrind --leak-check=full ./eduos` and paste the final summary in your README. Any leaked blocks will cost marks.
- **Makefile:** Targets: `all`, `clean`, `race`, `fixed`, `memcheck`. No hard-coded absolute paths.
- Error handling: every system call return value must be checked; use `perror()` appropriately.
- No external libraries for scheduling logic — write all algorithms yourself.

Part 3: Scheduling Simulator in Python

| File: `scheduler_sim.py` | Python 3.8+ | `pip install -r requirements.txt`

3.1 Scheduling Algorithms

Implement the following four algorithms. Each must be a self-contained function that accepts a list of process dictionaries and returns a schedule (ordered list of `[pid, start, end]` tuples):

- **FCFS** — First Come, First Served (non-preemptive). Handle ties in `arrival_time` by lower PID first.
- **SJF** — Shortest Job First (non-preemptive). For equal `burst_times`, fall back to FCFS order.
- **Priority Scheduling** — Non-preemptive; lower priority number = higher urgency. Implement **ageing**: every 3 time units a waiting process gains +1 priority to prevent starvation. Document the ageing logic clearly.
- **Round Robin** — Preemptive. User-defined time quantum (CLI flag: `--quantum N`). Maintain a proper ready queue. Demonstrate behaviour with at least two different quantum values in your report.

3.2 Input / Output

Your simulator must accept processes from two sources:

- **Random generation:** `python scheduler_sim.py --random 10` — generates 10 processes with randomised (but reproducible with `--seed N`) arrival, burst, and priority values.

OS - ASSIGNMENT - CS & IT DEPARTMENT

- **File input:** `python scheduler_sim.py --file sample_processes.csv` — reads CSV or JSON; the exact schema must be documented in your README. Provide a sample file in the repo.
- **PCB bridge:** Also accept `pcb_snapshot.json` written by your C process manager (Part 2 integration).

For every algorithm run, print a results table including:

- **Per-process:** Arrival Time, Burst Time, Completion Time, Turnaround Time (TAT), Waiting Time (WT), Response Time (RT).
- **Aggregate:** Average WT, Average TAT, Average RT, CPU Utilisation (%), Throughput (processes/unit time).

3.3 Visualisation

Use matplotlib (or seaborn) to produce the following; all charts must be saved as .png files in docs/screenshots/:

- **Gantt Chart** (per algorithm) — horizontal bar chart with process colours, time-axis ticks at every unit, and a legend. Idle CPU gaps must be shown in grey.
- **Comparison Bar Charts** — side-by-side comparison of all four algorithms for Average WT, Average TAT, and CPU Utilisation.
- **Timeline Animation** (bonus-level quality) — use matplotlib.animation to step through the Gantt chart one time unit at a time. Optional but rewarded under bonus.
- **Comparison Table** — print a Rich / tabulate formatted table to stdout with all metrics for all four algorithms side-by-side.

3.4 Thread Scheduling Simulation

Extend `scheduler_sim.py` to accept a `--mode thread` flag. In thread mode, each 'process' represents a thread within a process, and the scheduler must respect thread group affinity (threads from the same PID are labelled together on the Gantt chart). Demonstrate how the scheduler behaves differently when threading overhead (context-switch cost) is factored in.

Part 4: Integration & System Demonstration

| File: `controller/main_controller.py`

4.1 Main Controller

Write `main_controller.py` that acts as the orchestration layer:

1. **Launch C simulator:** Use Python's `subprocess.Popen` to start the compiled C binary. Capture stdout in real-time (not via wait).
2. **Feed processes:** Generate or read a process list; write it to a JSON file; signal the C binary via stdin to load it.

OS - ASSIGNMENT - CS & IT DEPARTMENT

3. **Collect results:** Monitor `pcb_snapshot.json` for updates; detect when all processes are TERMINATED.
4. **Hand off to Python scheduler:** Pass the completed PCB snapshot directly to `scheduler_sim.py` and trigger all four algorithms.
5. **Generate summary report:** Write a timestamped `simulation_report.json` containing metrics from all four algorithms.

4.2 OS Concept Demonstration

Your controller's README section and report must explicitly explain (with code references) how your implementation demonstrates the following Unit I concepts:

- **System Calls:** Map `edu_fork/exec/wait/exit` to real Linux system calls (`fork(2)`, `execve(2)`, `wait(2)`, `_exit(2)`). Explain what the kernel does that your simulation simplifies.
- **OS Structure:** Identify which parts of EduOS correspond to kernel space vs user space. Describe how a monolithic vs microkernel design would change your architecture.
- **Protection & Security:** Your shared memory IPC must implement an access-control check: a process may only read/write shared memory if its PCB contains a matching `owner_id` field. Explain how this relates to OS protection rings.
- **Virtual Machine Concept:** Write a Slide section in your PPT explaining how your PCB-isolated memory spaces mirror Type-2 hypervisor guest isolation.

Part 5: PRESENTATION

10 Marks | Max 15 SLIDES (excluding TITLE SLIDE) | Submit as PPT

Your PPT must contain the following sections (use these exact headings):

6. **Abstract** (≤150 words) — what you built and what you learned.
7. **Design Decisions** — Why this data structure for the PCB? Why POSIX shared memory over message queues for your IPC? Justify at least four major design choices with reference to OS theory.
8. **Syllabus Mapping Table** — A table with three columns: Syllabus Topic | How EduOS Implements It | File / Line Reference. Must cover every bullet point from the Unit I and Unit II overviews in Part 1.
9. **Scheduling Results & Analysis** — Include Gantt charts for all four algorithms on the same process set. Interpret the results: which algorithm gives the lowest average waiting time for your dataset? Why? When would a different algorithm win?
10. **Threading Issues Encountered** — Describe your race condition demonstration: what went wrong, what the valgrind / thread sanitizer output showed, and exactly how the mutex fixed it.
11. **C vs Python: Lessons Learned** — Reflect on the experience of implementing the same concept at two abstraction levels (e.g., a ready queue in C vs Python). At least 200 words.
12. **Challenges & Solutions** — At least three specific technical challenges (not 'it was hard') with the approach you took to resolve each.
13. **GitHub Summary** — Repository URL, total commits, brief description of branching strategy, and a paste of `git log --online` for the last 15 commits.

OS - ASSIGNMENT - CS & IT DEPARTMENT

Evaluation Summary

Component	Criteria	Marks
Part 1: GitHub	Repository structure, README quality, commit history, .gitignore	10
Part 2: C Core	Process struct & PCB table, edu_fork/exec/wait/exit, thread pool, threading models, race condition + fix, IPC, memory management (valgrind)	35
Part 3: Python Scheduler	All 4 algorithms correct, metrics (AWT/ATT/throughput), Gantt chart quality, comparison table, CSV/JSON input support	30
Part 4: Integration	main controller, subprocess bridge, stdout/JSON handoff, Unit I/II concept demonstration	15
Part 5: Report	Design rationale, syllabus mapping, analysis of results, screenshots, GitHub summary	10

Submission Checklist

- ☐ GitHub repo named correctly and accessible to instructor
- ☐ README.md complete with all required sections
- ☐ C code compiles with zero warnings; Makefile has all targets
- ☐ Valgrind summary pasted in README
- ☐ Race condition demo (make race vs make fixed) documented with output
- ☐ All 4 scheduling algorithms implemented and tested
- ☐ Gantt charts + comparison charts saved as .png in docs/screenshots/
- ☐ main_controller.py runs end-to-end without error
- ☐ report.pdf uploaded to docs/ and linked in README
- ☐ At least 10 commits with meaningful messages
- ☐ requirements.txt present in python_scheduler/
- ☐ Bonus components (if any) listed in README Bonus section